

**UNIVERSIDADE ESTADUAL DO SUDOESTE DA BAHIA
CIÊNCIA DA COMPUTAÇÃO
DESENVOLVIMENTO WEB 2026.1**

**Thiago Fernandes
Vinícius de Oliveira**

**MANUAL TÉCNICO DE INSTALAÇÃO E ARQUITETURA
SISTEMA DE PRÉ-MATRÍCULA ACADÊMICA**

**Vitória da Conquista
2026**

**THIAGO FERNANDES
VINÍCIUS DE OLIVEIRA**

MANUAL TÉCNICO DE INSTALAÇÃO E ARQUITETURA
SISTEMA DE PRÉ-MATRÍCULA ACADÊMICA

Manual de implantação, configuração, análise estrutural e engenharia de software do Sistema de Pré-Matrícula Acadêmica, desenvolvido como requisito avaliativo prático para a disciplina de Desenvolvimento Web do curso de Bacharelado em Ciência da Computação da Universidade Estadual do Sudoeste da Bahia (UESB).

Vitória da Conquista
2026

SUMÁRIO

1	INTRODUÇÃO	2
1.1	Objetivo	2
1.2	Público-alvo	2
1.3	Escopo Técnico	2
2	TECNOLOGIAS UTILIZADAS	3
2.1	Camada de Segurança Integrada	3
3	ESTRUTURA DO PROJETO	4
3.1	Visualização da Estrutura do Backend	4
3.2	Visualização da Estrutura do Frontend	5
4	COMO EXECUTAR O SISTEMA	6
4.1	Passo 1: Subir o Banco de Dados (PostgreSQL)	6
4.2	Passo 2: Configurar e Iniciar o Backend (Spring Boot)	6
4.3	Passo 3: Instalar e Iniciar o Frontend (Angular)	7
4.4	Passo 4: Validação do Ambiente	7
5	CONCLUSÃO	8

1 INTRODUÇÃO

1.1 Objetivo

Este manual apresenta as diretrizes técnicas necessárias para compreender, configurar, instalar, executar e manter o Sistema de Pré-Matrícula Acadêmica. Este guia consolida as especificações da pilha tecnológica de desenvolvimento, os padrões arquiteturais de código-fonte, a topologia de banco de dados adotada e o fluxo dinâmico de comunicação do software corporativo.

1.2 Público-alvo

Este documento destina-se a desenvolvedores de software encarregados do suporte evolutivo do sistema, professores avaliadores de conformidade da arquitetura, equipes de operações e integradores de infraestrutura local de bancos de dados relacionais e de orquestração de containers Docker.

1.3 Escopo Técnico

O Sistema de Pré-Matrícula Acadêmica baseia-se em uma arquitetura cliente-servidor, sendo composto por:

- **Apresentação:** Aplicação *Single Page Application* (SPA) desenvolvida em Angular com TypeScript e Tailwind CSS.
- **Serviço/API REST:** Aplicação backend baseada no ecossistema Java 21 e Spring Boot.
- **Persistência e Armazenamento:** Banco de dados relacional PostgreSQL executado em container Docker, com versionamento e evolução de esquema estruturados pelo Flyway Migration.

2 TECNOLOGIAS UTILIZADAS

O sistema foi concebido utilizando ferramentas consagradas de engenharia de software para garantir escalabilidade horizontal, fácil manutenibilidade de domínio e segurança nas transações acadêmicas discentes. A Tabela 1 detalha a distribuição das tecnologias em uma grade estruturada.

Tabela 1: Detalhamento da pilha tecnológica e responsabilidades

Tecnologia	Escopo de Uso	Responsabilidade Técnica
Java 21	Backend Core	Utilização de <i>Records</i> , novas APIs de concorrência e tipagem forte.
Spring Boot 3.x	API REST	Injeção de dependência nativa, gestão transacional e Web MVC.
Spring Data JPA	Persistência	Camada de abstração e Repositories de acesso ao PostgreSQL.
Hibernate	Mapeamento ORM	Tradução objeto-relacional estruturada do modelo físico.
Spring Security	Segurança	Barreira de filtros de segurança, autenticação e políticas RBAC.
JWT (JSON Token)	Autorização	Sessões <i>stateless</i> assinadas digitalmente para requisições HTTPS.
BCrypt	Criptografia	Algoritmo de hash salt para armazenamento seguro de senhas.
Bean Validation	Validação	Filtro declarativo de DTOs via anotações <i>jakarta.validation</i> .
Flyway Migration	Banco de Dados	Versionamento autônomo do esquema do banco via scripts SQL.
PostgreSQL 15	Armazenamento	Banco de dados relacional com integridade referencial nativa.
Docker / Compose	Infraestrutura	Isolamento local do SGBD e do console pgAdmin em containers.
Angular 17+	Frontend SPA	Interface com componentes <i>standalone</i> , rotas lazily loaded e RxJS.
Tailwind CSS	Estilização	Framework utilitário util-first para responsividade fluida ABNT.
jsPDF / AutoTable	Relatórios	Exportação direta e cliente-side de relatórios operacionais em PDF.
OpenAPI / Swagger	Documentação	Geração interativa automática do catálogo e testes da API REST.

2.1 Camada de Segurança Integrada

A autenticação do sistema ocorre de maneira assíncrona baseada em tokens JWT. As senhas informadas pelo usuário durante o cadastro passam por um algoritmo de codificação unidirecional (BCrypt) antes de serem de fato inseridas na base de dados, prevenindo qualquer tipo de vazamento ou exposição. Toda requisição a rotas privadas exige o envio do token JWT no cabeçalho HTTP de requisição no formato `Authorization: Bearer <token-jwt>`.

3 ESTRUTURA DO PROJETO

O ecossistema adota o formato de *monorepo*, dividindo o código-fonte nas pastas exclusivas do backend, frontend e docs.

No backend, os pacotes são estruturados sob a perspectiva de **arquitetura baseada em domínios** (*Domain Driven Package Structure*), diminuindo o acoplamento sistêmico e permitindo que cada domínio gerencie seus próprios Controllers, Services e Repositories. No frontend, adota-se a convenção **feature-first**, segmentando as funcionalidades por grupos de atores (admin, student) e infraestrutura (core, shared).

3.1 Visualização da Estrutura do Backend

A estrutura do projeto Java no Spring Boot está organizada conforme o mapeamento a seguir:

```
backend/  
  src/  
    main/  
      java/br/edu/uesb/prematricula/  
        academicperiod/      # Cadastro e controle de semestres letivos  
        controller/          # Endpoints HTTP REST expostos  
        exception/           # Exceções exclusivas deste domínio  
        model/  
          dto/                # Objetos de transferência de dados (DTOs)  
          entity/            # Modelo de persistência JPA/Hibernate  
          repository/        # Interfaces de repositório Spring Data  
          service/           # Concentração das regras de negócio (Services)  
        admin/               # Controle cadastral de administradores  
        auth/                # Autenticação e fluxo de primeiro acesso  
        classgroup/          # Cadastro de turmas, vagas e excedentes  
        config/security/     # Filtros JWT, Spring Security e UserDetails  
        discipline/          # Cadastro e catálogo de disciplinas  
        enrollment/          # Criação, cancelamento e atualização de matrículas  
        enrollmentprocess/    # Janelas de abertura e encerramento de processos  
        reports/             # Geração de consultas e relatórios gerenciais  
        student/             # Controle cadastral de discentes  
        user/                # Dados comuns de usuários e credenciais  
        verificationtoken/    # Geração e consumo de tokens de primeiro acesso  
      resources/  
        db/migration/        # Migrações Flyway organizadas por cronologia  
        application.yml      # Parametrização e conexões da aplicação
```

3.2 Visualização da Estrutura do Frontend

A aplicação Angular no frontend adota a seguinte organização lógica:

```
frontend/  
  src/  
    app/  
      core/          # Serviços e infraestrutura singleton global  
        guards/      # Guardas de rotas baseadas em perfis (RBAC)  
        interceptors/ # Inclusão automática de JWT nas requisições  
        models/      # Interfaces TypeScript e contratos de dados  
        services/    # Serviços compartilhados (Autenticação, etc.)  
      features/      # Módulos e telas de regras de negócio  
        admin/       # Funcionalidades exclusivas do Administrador  
          academicperiod/ # Gestão do período letivo  
          classgroup/   # Gestão das turmas ofertadas  
          dashboard/   # Métricas e indicadores em tempo real  
          discipline/  # Gestão do portfólio de disciplinas  
          reports/     # Consultas de demandas de matrículas  
          student/     # Cadastro e gestão de discentes  
        auth/        # Telas de login e primeiro acesso  
        student/     # Funcionalidades exclusivas do Discente  
          dashboard/  # Painel de controle do aluno  
          enrollmentprocess/ # Fluxo de pré-matrícula e alterações  
          myhistory/  # Acesso ao histórico acadêmico discente  
      layout/        # Layouts e templates de páginas  
        admin-layout/ # Invólucro com menus do administrador  
        auth-layout/  # Tela limpa de login  
        student-layout/ # Painel e sidebar discente  
      shared/        # Componentes genéricos compartilhados  
        components/sidebar/ # Sidebar dinâmica baseada em perfil
```

4 COMO EXECUTAR O SISTEMA

Para a inicialização do ambiente local de desenvolvimento, certifique-se de possuir em seu host os seguintes requisitos de ambiente ativos: Git, Docker, JDK 21 e Node.js 22.x.

4.1 Passo 1: Subir o Banco de Dados (PostgreSQL)

O banco de dados relacional e sua respectiva ferramenta de monitoramento administrativo (pgAdmin) são inicializados em segundo plano isolados através de containers Docker.

Na raiz do repositório, execute o comando correspondente:

```
docker compose up -d
```

Para certificar que os serviços estão rodando em conformidade na porta local declarada (5435), execute:

```
docker compose ps
```

4.2 Passo 2: Configurar e Iniciar o Backend (Spring Boot)

Ao ser iniciado, o Spring Boot acionará o Flyway de forma integrada para avaliar o histórico de migrations aplicadas no PostgreSQL. Ele criará de maneira automática todas as tabelas, índices e chaves estrangeiras necessárias para a operação do sistema.

Navegue até o diretório do backend e execute a instrução dependente do seu sistema operacional:

Em sistemas Microsoft Windows:

```
cd backend
.\mvnw.cmd spring-boot:run
```

Em sistemas Linux ou macOS:

```
cd backend
chmod +x mvnw
./mvnw spring-boot:run
```

Assim que o processo do Spring Framework for totalmente inicializado, a API REST e a documentação interativa das rotas estarão ativas sob o seguinte endereço eletrônico local: `http://localhost:8080`.

4.3 Passo 3: Instalar e Iniciar o Frontend (Angular)

Para carregar as dependências de interface e iniciar o servidor dinâmico de desenvolvimento do Angular, abra um novo terminal, entre na pasta do frontend e execute os comandos:

```
cd frontend
npm install
npm start
```

Após a inicialização do compilador, a aplicação SPA estará aberta aceitando requisições no navegador em: <http://localhost:4200>.

4.4 Passo 4: Validação do Ambiente

A Tabela 2 resume os endereços importantes que podem ser acessados em ambiente de desenvolvimento local para fins de validação e testes integrados.

Tabela 2: Endereços de validação da infraestrutura local

Serviço	Porta Local	Endereço de Acesso no Navegador
Frontend Angular	4200	http://localhost:4200 (Tela de Login principal)
Backend Spring Boot	8080	http://localhost:8080 (Raiz de Endpoints)
Documentação Swagger	8080	http://localhost:8080/swagger-ui/index.html
pgAdmin Console	5052	http://localhost:5052 (Administração do Banco)

5 CONCLUSAO

O Sistema de Pré-Matrícula Acadêmica foi estruturado utilizando padrões rígidos de mercado de desenvolvimento de software de alta fidelidade. O acoplamento lógico isolado por pacotes de domínios no backend Java com Spring Boot, em consonância com a arquitetura baseada em features do frontend Angular, garante que novos requisitos ou fluxos possam ser expandidos sem degradar o ecossistema existente.

A correta aplicação e observância dos passos descritos neste Manual Técnico garante que qualquer desenvolvedor ou auditor possa reerguer o ambiente de forma isolada, monitorar a escrita do PostgreSQL por migrações auto-geridas e interagir com o fluxo funcional homologado para o semestre ****2026.1****.